Software Engineering

Chapter 6 Architectural Design
Chapter 7 Design and Implementation

Topics covered

- Architectural Design
  - Architectural patterns
  - Application architectures
- Object-oriented design using the UML
- Design patterns
- Implementation issues
- Open source development

Architectural Design

Architectural design

- Architectural design is concerned with understanding how a software system should be organized and,
  - ***designing the overall structure of that system***.
- Architectural design is the critical link between design and requirements engineering,
  - as it ***identifies the main structural components in a system and the relationships between them***.
- It is generally accepted that an early stage of agile processes is to design an overall systems architecture.
- ***Refactoring the system architecture is usually expensive*** because it affects so many components in the system

Architectural abstraction

- Architecture in the small is concerned with ***the architecture of individual programs***.
  - At this level, we are concerned with the way that an individual program is decomposed into components.
- Architecture in the large is concerned with ***the architecture of complex enterprise systems***,
  - that include other systems, programs, and program components.
  - These enterprise systems are distributed over different computers, which may be owned and managed by different companies.

Advantages of explicit architecture

- **Stakeholder communication**
  - Architecture may be used as a focus of discussion by system stakeholders.
- **System analysis**
  - Means that analysis of whether the system can meet its non-functional requirements is possible.
- **Large-scale reuse**
  - The architecture may be reusable across a range of systems
  - Product-line architectures may be developed.

Architectural design decisions

- Architectural design is a creative process so ***the process differs depending on the type of system being developed***.
- However, a number of common decisions span all design processes, and these decisions affect the non-functional characteristics of the system.
  - e.g.) Is there a generic application architecture used as a template?
  - What architectural patterns or styles can be used?
  - How will the structural components in the system be decomposed into sub-components?

Architecture reuse

- Systems in the same domain often have similar architectures that reflect domain concepts.
- Application product lines are built around a core architecture with variants that satisfy particular customer requirements.
- The architecture of a system may be designed around one of more architectural patterns or ‘styles’.
  - These capture the essence of an architecture and can be instantiated in different ways.

Application architectures

- Application systems are designed to meet an organisational need.
- As businesses have much in common, their application systems also tend to have *a common architecture that reflects the application requirements*.
- A ***generic application architecture*** is an architecture for a type of software system that may be configured and adapted to create a system that meets specific requirements.

Architecture and system characteristics

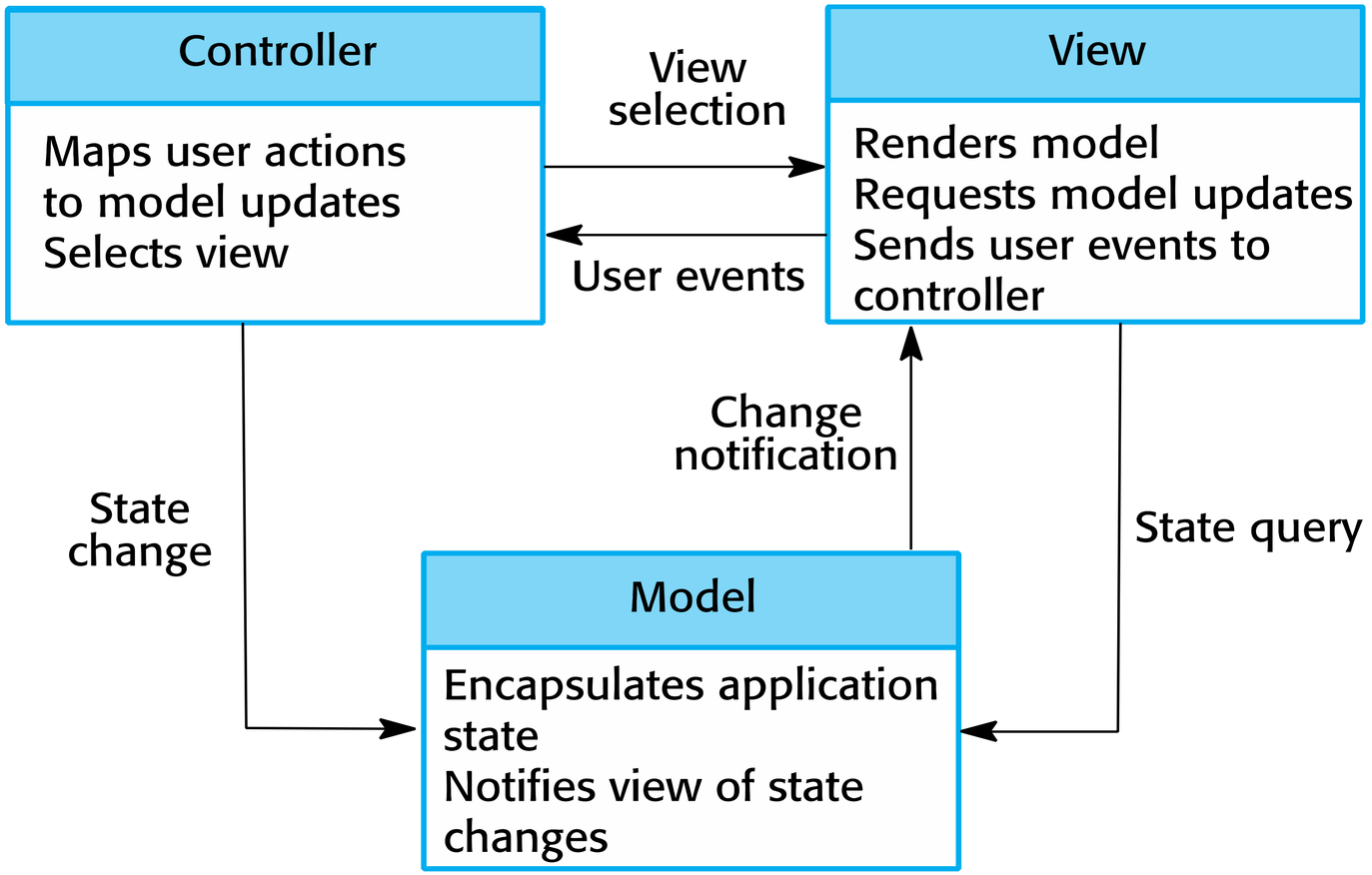
- **Performance**: Localise critical operations and minimise communications. Use large rather than fine-grain components.
- **Security**: Use a layered architecture with critical assets in the inner layers.
- **Safety**: Localise safety-critical features in a small number of sub-systems.
- **Availability**: Include redundant components and mechanisms for fault tolerance.
- **Maintainability**: Use fine-grain, replaceable components.

Architectural patterns

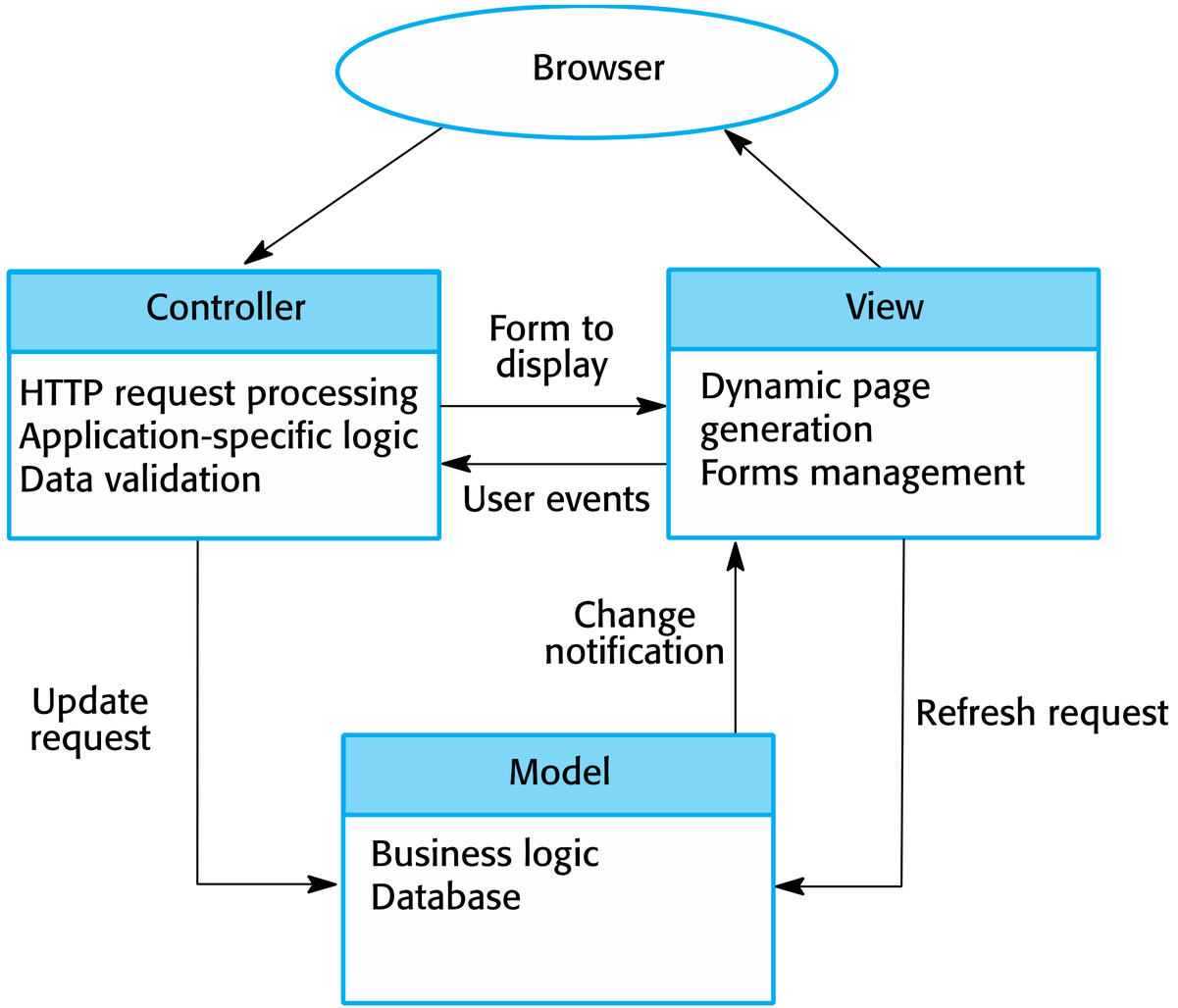
Architectural patterns

- Patterns are a means of representing, sharing and reusing knowledge.
- ***An architectural pattern is a stylized description of good design practice***, which has been tried and tested in different environments.
- Patterns should include *information about when they are useful or not*.
- Patterns may be represented using tabular and graphical descriptions.
- It is important to understand *when and how a pattern is used*, and what are the *advantages and disadvantages*.

The organization of the Model-View-Controller



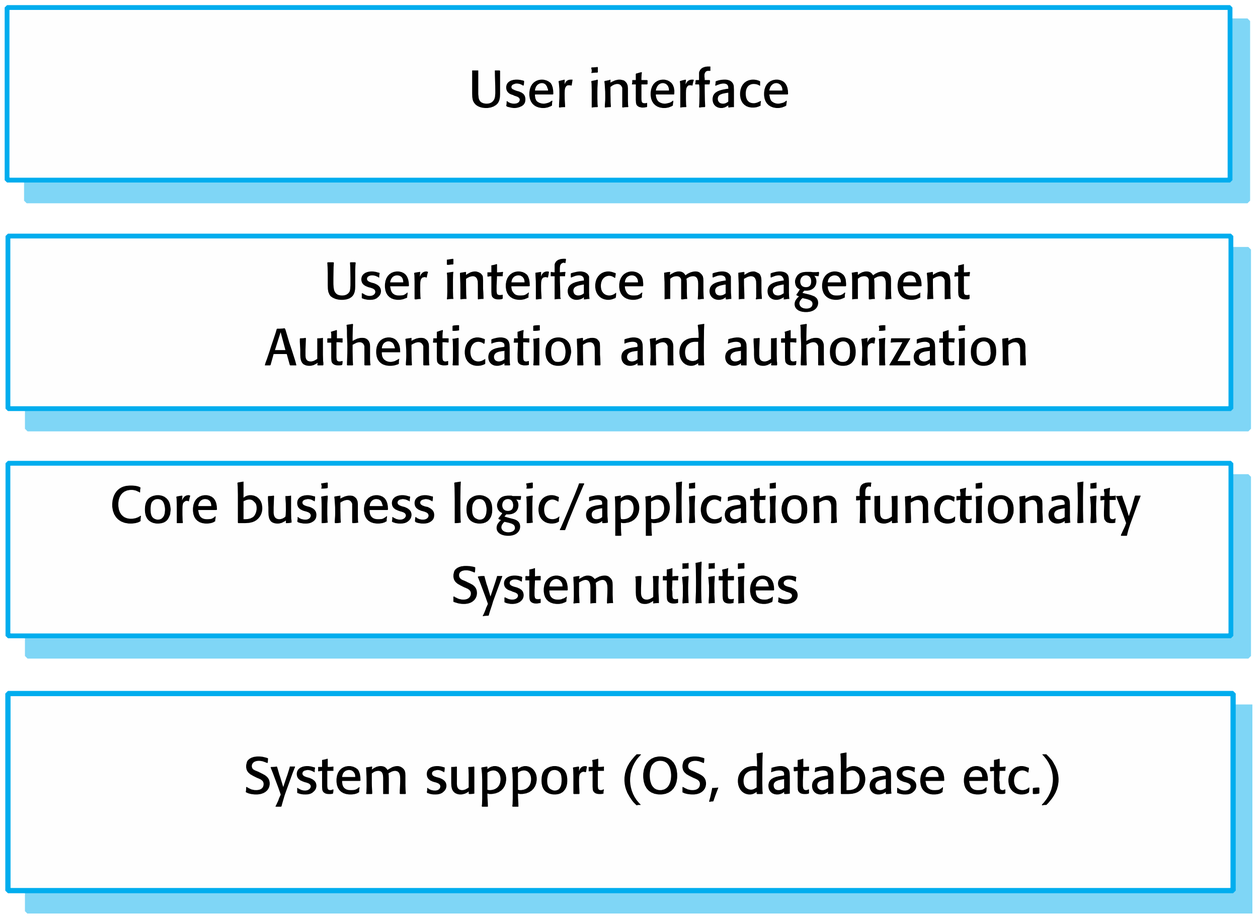
Web application architecture using the MVC pattern



Layered architecture

- Used to model the interfacing of sub-systems.
- Organises the system into a set of layers (or abstract machines) each of which provide a set of services.
- Supports the incremental development of sub-systems in different layers.
  - When a layer interface changes, only the adjacent layer is affected.
- However, often artificial to structure systems in this way.

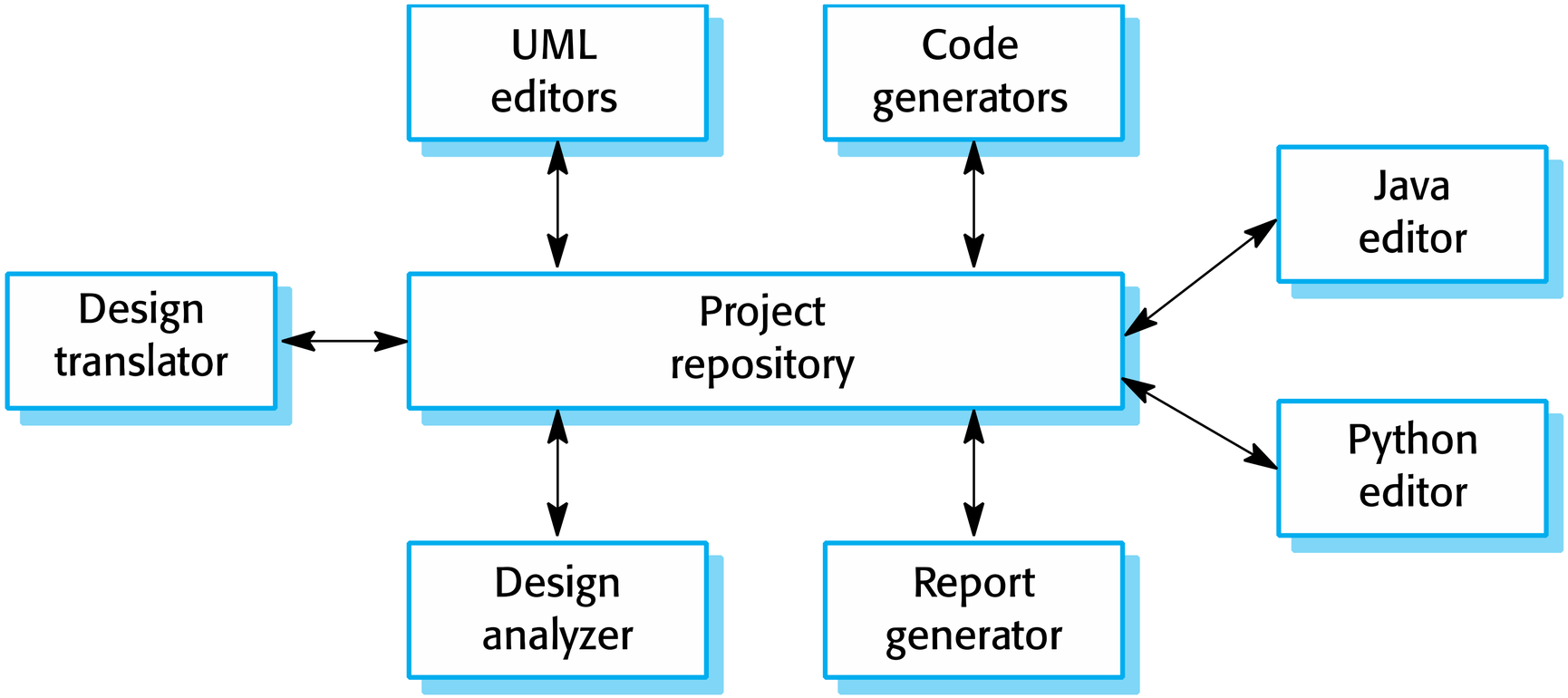
A generic layered architecture



Repository architecture

- Sub-systems must exchange data. This may be done in two ways:
  - Shared data is held in a central database or repository and may be accessed by all sub-systems;
  - Each sub-system maintains its own database and passes data explicitly to other sub-systems.
- When large amounts of data are to be shared, the repository model of sharing is most commonly used a this is an efficient data sharing mechanism.

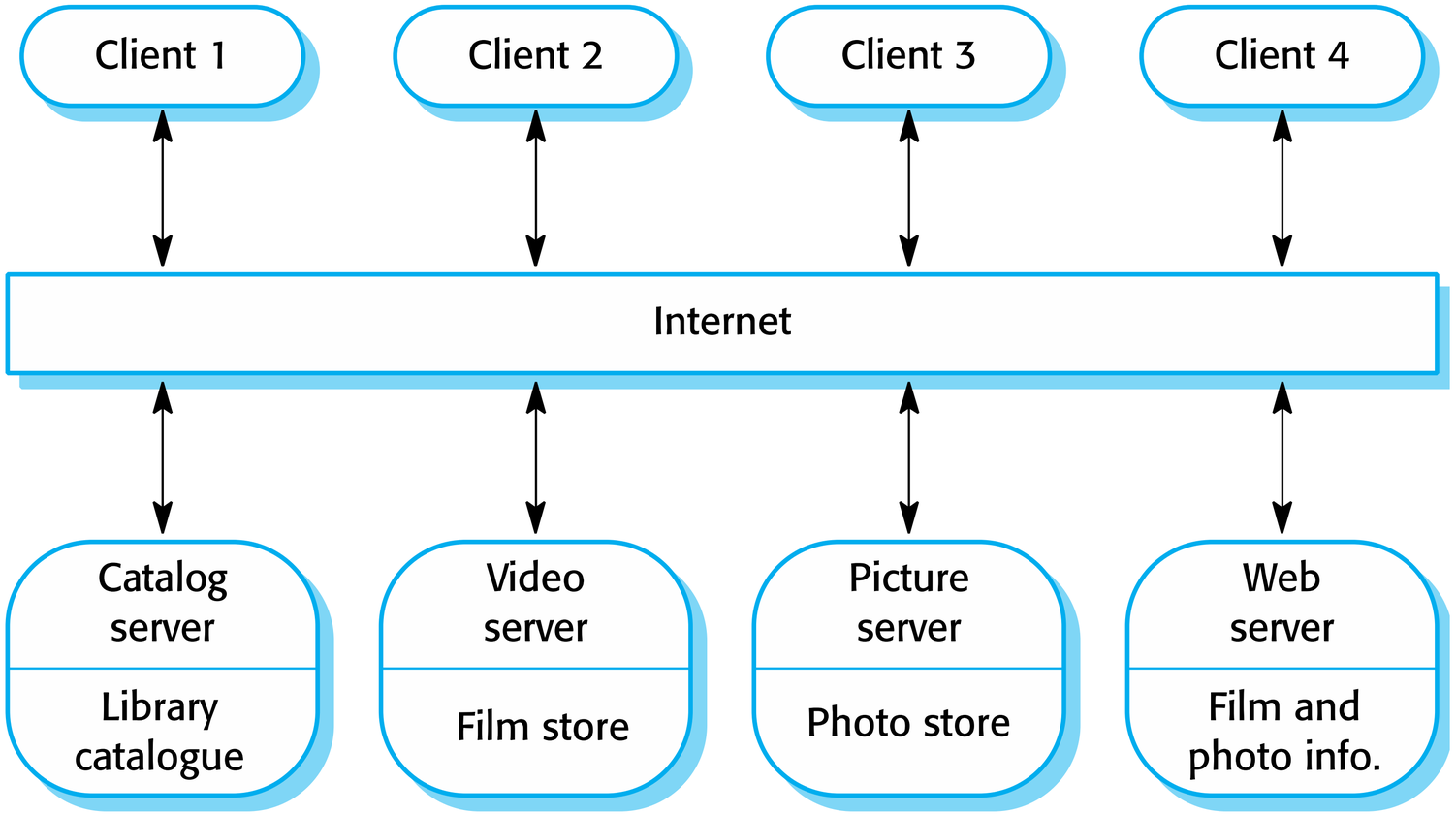
A repository architecture for an IDE



Client-server architecture

- Distributed system model which shows how data and processing is distributed across a range of components.
  - Can be implemented on a single computer.
- Set of stand-alone servers which provide specific services such as printing, data management, etc.
- Set of clients which call on these services.
- Network which allows clients to access servers.

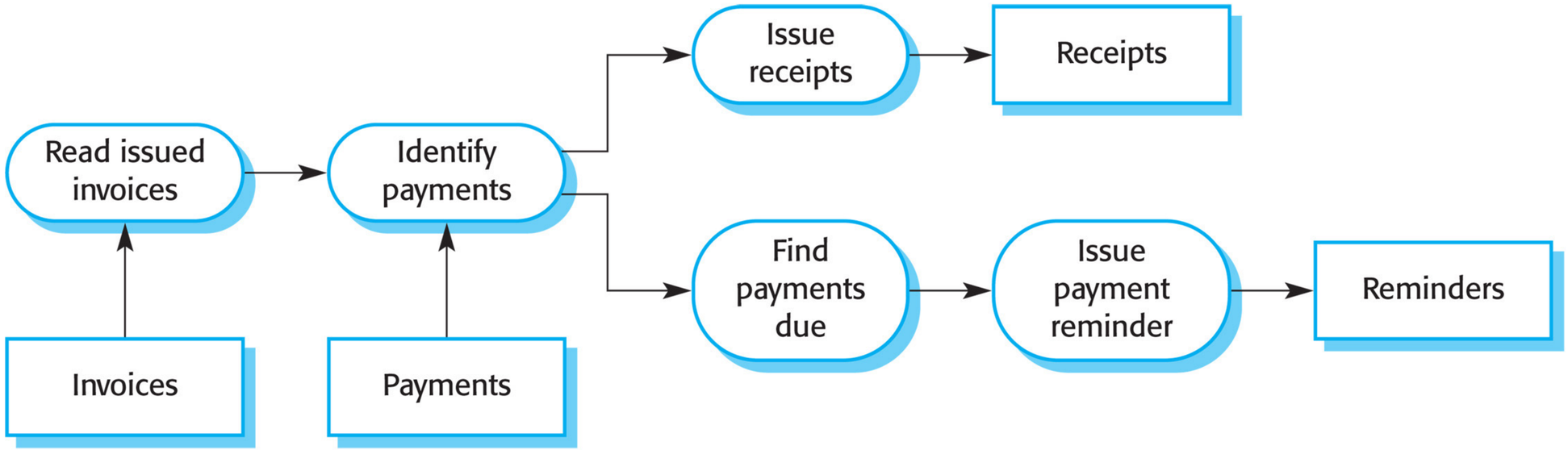
A client–server architecture for a film library



Pipe and filter architecture

- Functional transformations process their inputs to produce outputs.
- May be referred to as a pipe and filter model (as in UNIX shell).
  - e.g.) `cat 2021-03-31.log | grep "error" | wc -l`
- Variants of this approach are very common.
  - When transformations are sequential, this is a batch sequential model which is extensively used in data processing systems.
- Not really suitable for interactive systems.

An example of the pipe and filter architecture used in a payments system



Design and Implementation

Design and implementation

- Software design and implementation is the stage in the software engineering process at which an executable software system is developed.
- Software design and implementation activities are invariably inter-leaved.
  - Software design is a creative activity in which you identify software components and their relationships, based on a customer’s requirements.
  - Implementation is the process of realizing the design as a program.

Build or buy

- In a wide range of domains, it is now possible to *buy off-the-shelf systems (COTS) that can be adapted and tailored to the users' requirements*.
- For example, if you want to implement a chatbot for customer services, you may buy a package or solution providing the service.
  - It can be cheaper and faster to use this approach rather than developing a system in a conventional programming language.
  - Nowadays, software sometimes comes with data, which require a lot of time to collect.
- When you develop an application in this way, the design process becomes concerned with how to use the configuration features of that system to deliver the system requirements.

Object-oriented design

An object-oriented design process

- Structured object-oriented design processes involve developing a number of different system models.
- They require a lot of effort for development and maintenance of these models and, for small systems, this may not be cost-effective.
- However, for large systems developed by different groups, design models are an important communication mechanism.
- ***Unified Modeling Language (UML)*** is a way to represent system models using graphical notations.

Process stages

- There are a variety of different object-oriented design processes that depend on the organization using the process.
- Common activities in these processes include:
  - Define the context and modes of use of the system;
  - Design the system architecture;
  - Identify the principal system objects;
  - Develop design models;
  - Specify object interfaces.
- The textbook uses a weather station system as an example.

System context and interactions

- ***Understanding the relationships between the software that is being designed and its external environment*** is essential,
  - for deciding how to provide the required system functionality and how to structure the system to communicate with its environment.
- ***Understanding of the context*** also lets you establish the boundaries of the system.
  - Setting the system boundaries helps you decide *what features are implemented in the system* being designed and *what features are in other associated systems*.

Architectural design

- Once interactions between the system and its environment have been understood,
  - you use this information for designing the system architecture.
- You identify the major components that make up the system and their interactions,
  - and then may organize the components using an architectural pattern.

Object class identification

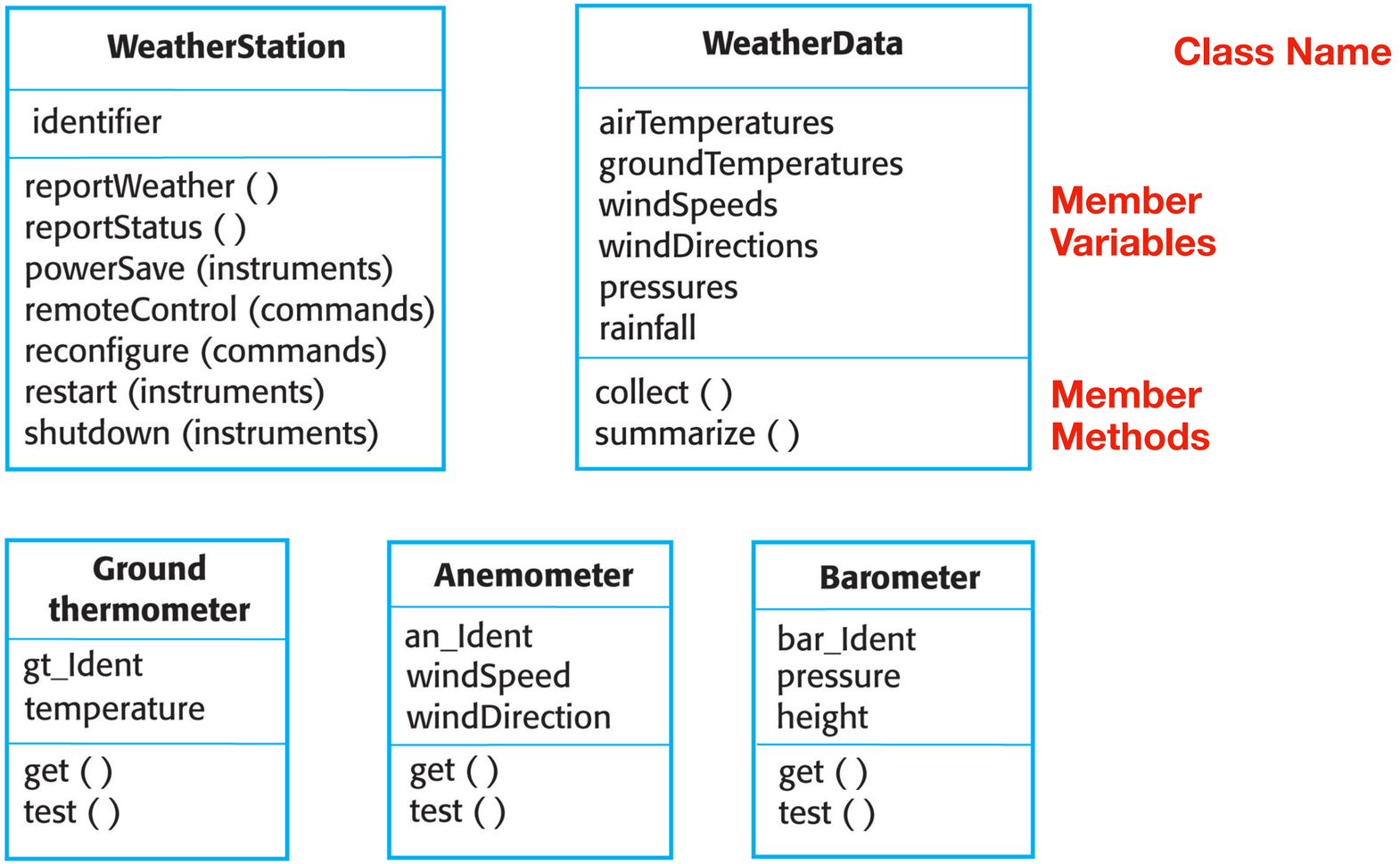
- Identifying object classes is often a difficult part of object oriented design.
- There is no 'magic formula' for object identification.
  - It relies on the skill, experience and domain knowledge of system designers.
- Object identification is an iterative process.
  - You are unlikely to get it right first time.
  - Continuous refactoring is always a good idea.

Approaches to identification

- Use a ***grammatical approach*** based on a natural language description of the system.
- ***Base the identification on tangible things*** in the application domain.
- Use a ***behavioural approach*** and identify objects based on what participates in what behaviour.
- Use a ***scenario-based analysis***. The objects, attributes and methods in each scenario are identified.
- Domain-Driven Design (DDD).
  - Design a model based on deep understanding of domain knowledge and make the model reflect business domain.

Weather station object classes

Class Diagrams



Design Activities w/ LLMs

To-Do App Example

- Here are the requirements of the To-Do App.
  - Users can register to-do items by clicking a button.
  - Each to-do item has title, description, and deadline, as well as a list of tags for additional information.
  - To-do items can be assigned to a certain user.
  - Users can register a list of sub-items to a to-do item.
  - To-do items are shown as a list, and users can sort them based on the title, deadline, or assignee.
  - To-do items can be filtered based on their tags too.
- Can you identify objects and their structures based on the requirements?

Design patterns

Design patterns

- A design pattern is a way of reusing abstract knowledge about a problem and its solution.
- A pattern is a description of the problem and the essence of its solution.
- It should be sufficiently abstract to be reused in different settings.
- Pattern descriptions usually make use of object-oriented characteristics such as inheritance and polymorphism.

Gang of Four (GoF)

- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.
- They wrote a book of a collection of design patterns.
- Design Patterns: Elements of Reusable Object-Oriented Software, 1995.
- Includes many design patterns about the design object-oriented software.



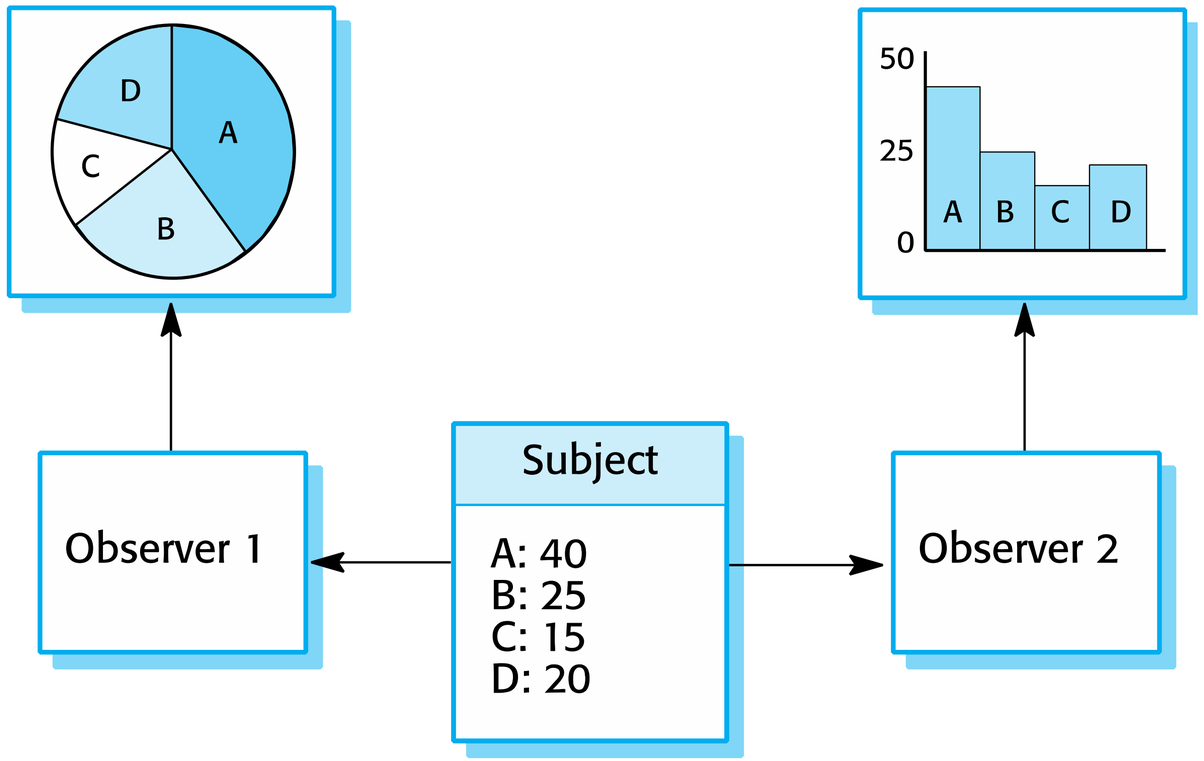
Pattern elements

- **Name**
  - A meaningful pattern identifier.
- **Problem** description.
- **Solution** description.
  - Not a concrete design but a template for a design solution that can be instantiated in different ways.
- **Consequences**
  - The results and trade-offs of applying the pattern.

The Observer pattern

- Name: Observer.
- Description: Separates the display of object state from the object itself.
- Problem description: Used when multiple displays of state are needed.
- Solution description: See slide with UML description.
- Consequences: Optimisations to enhance display performance are impractical.

Multiple displays using the Observer pattern



Design problems

- To use patterns in your design, you need to recognize that any design problem you are facing may have an associated pattern that can be applied.
  - Tell several objects that the state of some other object has changed (***Observer*** pattern).
  - Tidy up the interfaces to a number of related objects that have often been developed incrementally (***Façade*** pattern).
  - Provide a standard way of accessing the elements in a collection, irrespective of how that collection is implemented (***Iterator*** pattern).
  - Allow for the possibility of extending the functionality of an existing class at run-time (***Decorator*** pattern).
- Also ***Adaptor***, ***Visitor***, ***Strategy*** and ***Factory method*** patterns are worth to check.

Implementation issues

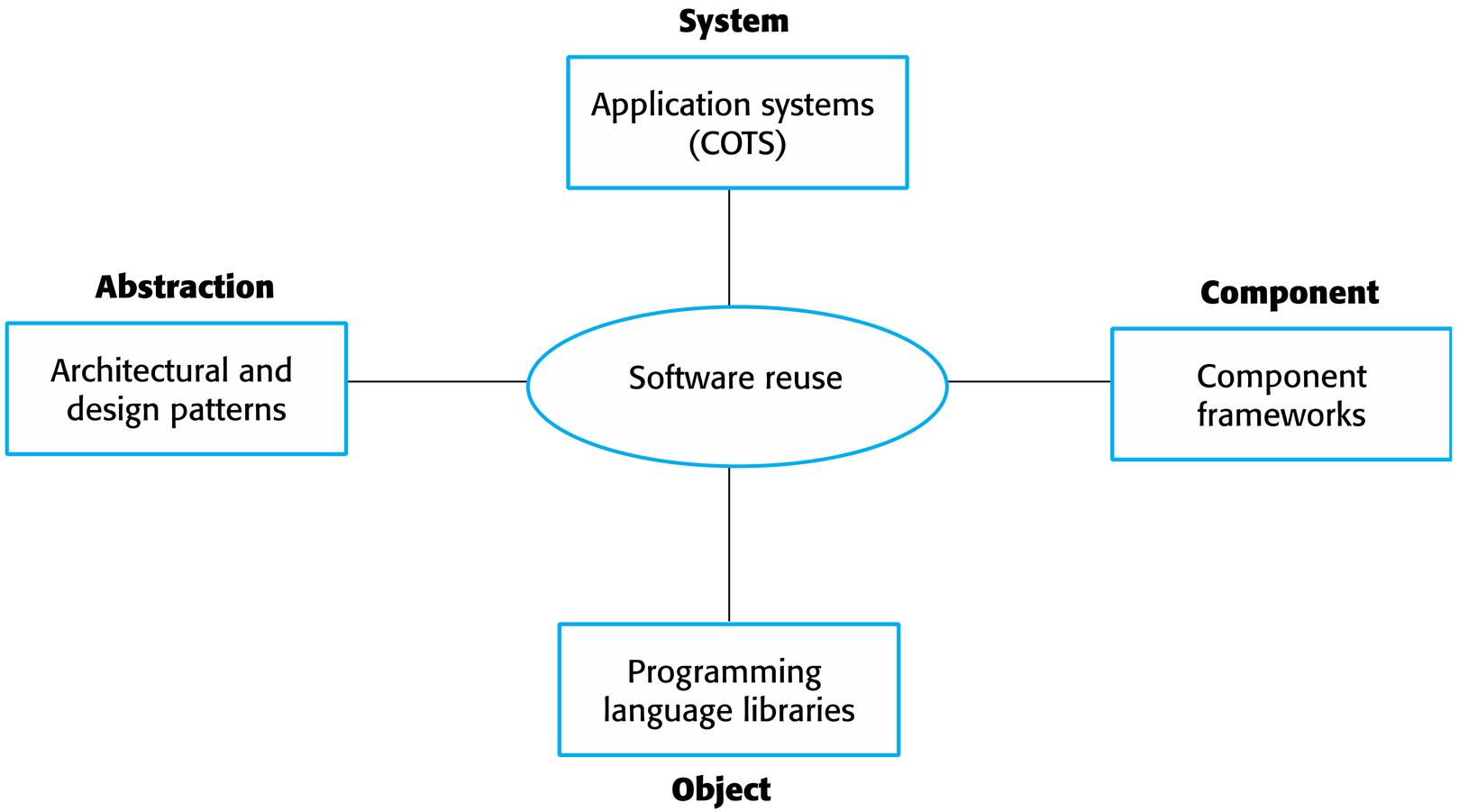
Implementation issues

- Focus here is not on programming, although this is obviously important, but on other implementation issues that are often not covered in programming texts:
  - **Reuse**: Most modern software is constructed by reusing existing components or systems.
  - **Configuration management**: During the development process, you have to keep track of the many different versions.
  - **Host-target development**: Production software is usually developed on one computer (the host system), and executed on a separate computer (the target system).

Reuse

- In the past, most new software was developed from scratch, by writing all code in a high-level programming language.
  - The only significant reuse or software was the reuse of functions and objects in programming language libraries.
- Costs and schedule pressure mean that this approach became increasingly unviable, especially for commercial and Internet-based systems.
- An approach to development based around the reuse of existing software emerged and is now generally used for business and scientific software.

Software reuse



Reuse costs

- The costs of the time spent in *looking for software* to reuse and *assessing whether or not it meets your needs*.
- Where applicable, the costs of *buying the reusable software*.
  - Nowadays, you often need to pay based on your transaction size.
- The costs of *adapting and configuring* the reusable software components or systems to reflect the requirements of the system that you are developing.
- The costs of *integrating reusable software elements* with each other (if you are using software from different sources) and with the new code that you have developed.

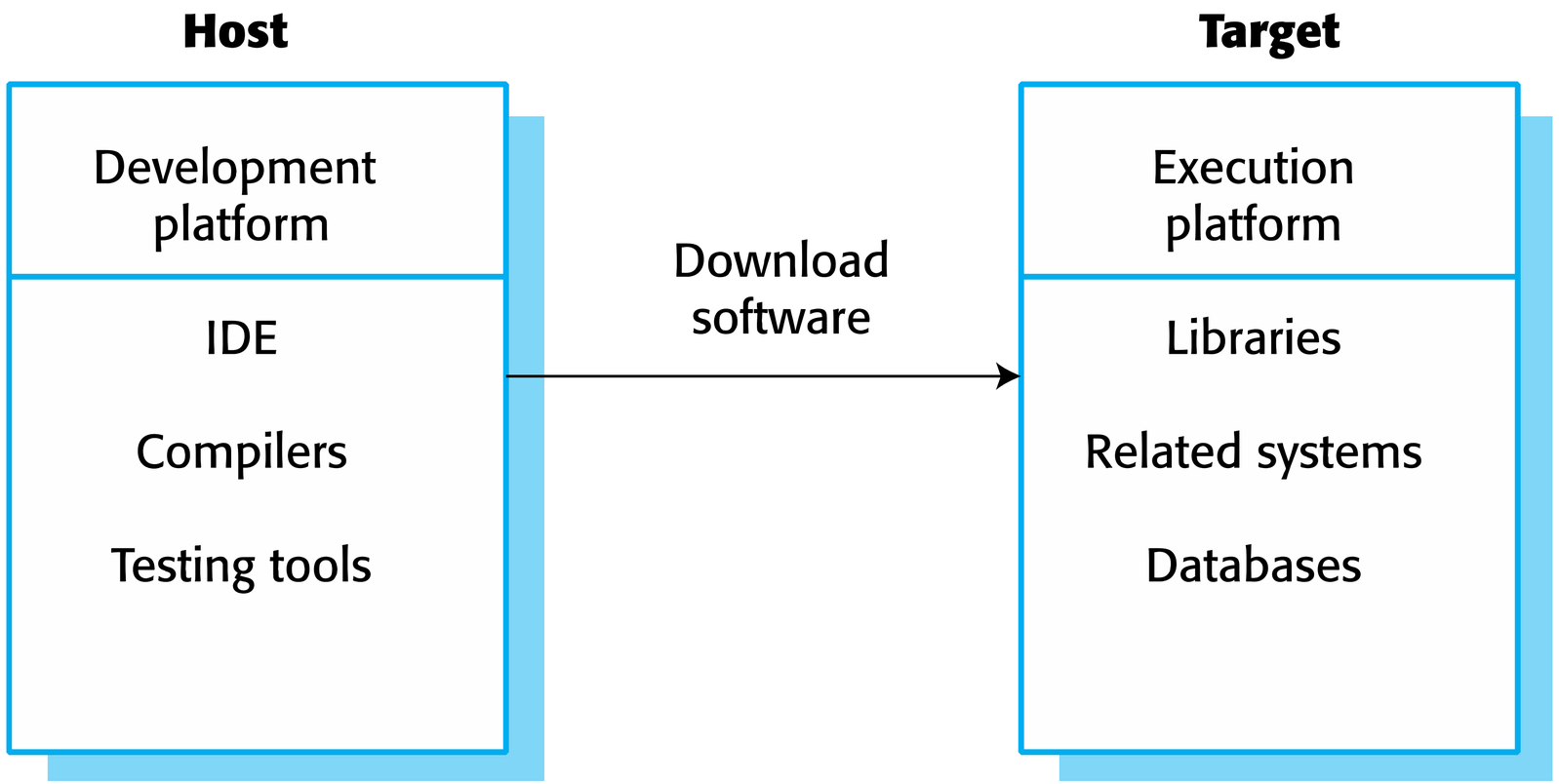
Configuration management

- Configuration management is the name given to the general process of managing a changing software system.
- The aim of configuration management is to support the system integration process,
  - so that all developers can access the project code and documents in a controlled way, find out what changes have been made, and compile and link components to create a system.
- We already discussed it in Week 5.

Host-target development

- Most software is developed on one computer (the host), but runs on a separate machine (the target).
- More generally, we can talk about a development platform and an execution platform.
  - A platform is more than just hardware.
  - It includes the installed operating system plus other supporting software such as a database management system or, for development platforms, an interactive development environment.
- Development platform usually has different installed software than execution platform; these platforms may have different architectures.

Host-target development



Development platform tools

- An integrated compiler and syntax-directed editing system that allows you to create, edit and compile code.
- A language debugging system.
- Graphical editing tools, such as tools to edit UML models.
- Testing tools, such as Junit that can automatically run a set of tests on a new version of a program.
- Project support tools that help you organize the code for different development projects.

Component/system deployment factors

- If a component is designed for a specific hardware architecture, or relies on some other software system,
  - it must obviously be deployed on a platform that provides the required hardware and software support.
- This can be supported by new tools and techniques, such as cloud platform services like AWS or container services like Docker.
- High availability systems may require components to be deployed on more than one platform.
  - This means that, in the event of platform failure, an alternative implementation of the component is available.

Component/system deployment factors

- If there is a high level of communications traffic between components, it usually makes sense to deploy them on the same platform or on platforms that are physically close to one other.
  - This reduces the delay between the time a message is sent by one component and received by another.
- For scalability, the same version of software may be deployed on multiple instances of platforms.
  - User traffic will be balanced on the instances, or even to instances with slightly different configurations.

Open source development

Open source development

- Open source development is an approach to software development in which the source code of a software system is published and volunteers are invited to participate in the development process.
- Its roots are in the Free Software Foundation (www.fsf.org), which advocates that source code should not be proprietary but rather should always be available for users to examine and modify as they wish.
- Open source software extended this idea by using the Internet to recruit a much larger population of volunteer developers.
    - Many of them are also users of the code.

Open source systems

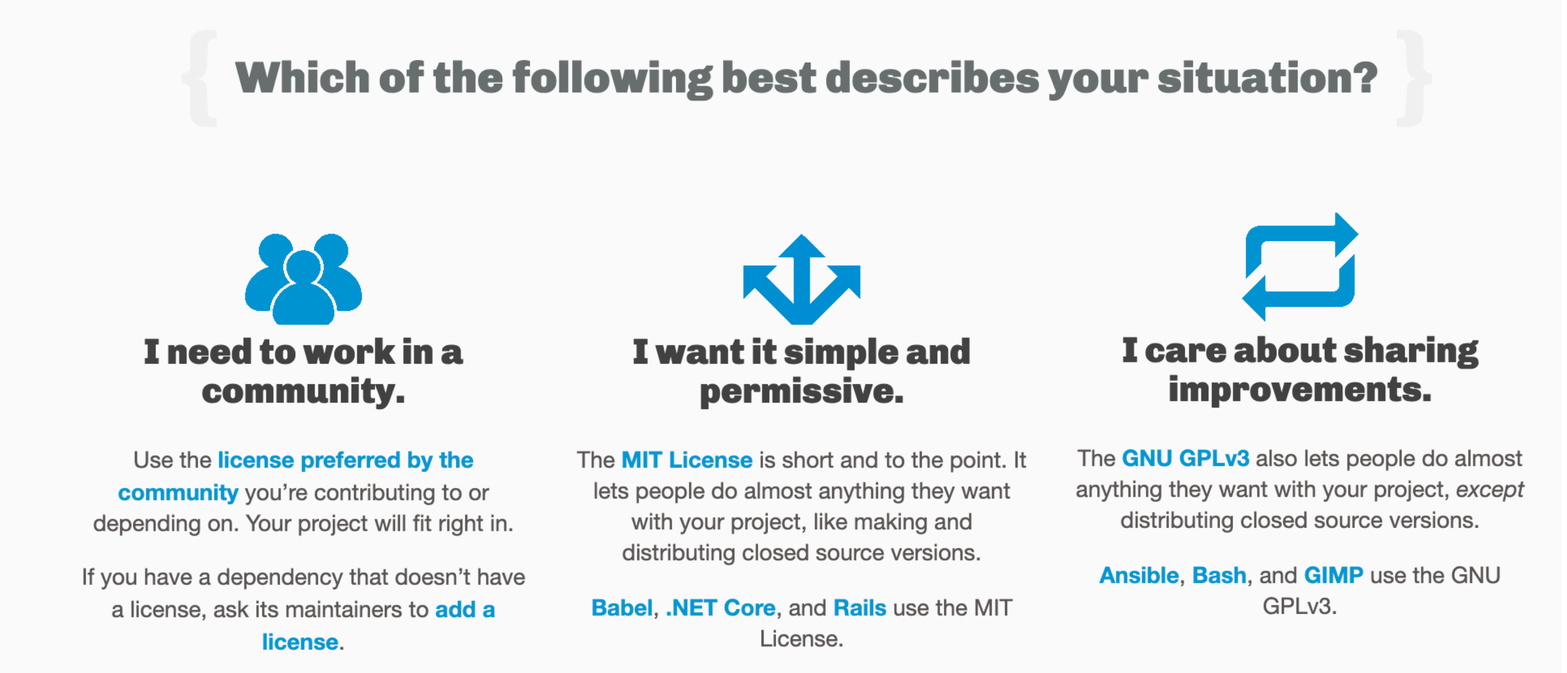
- The best-known open source product is, of course, the Linux operating system which is widely used as a server system and, increasingly, as a desktop environment.
- Other important open source products are Java, the Apache web server and the MySQL database management system.
  - Tensorflow, NumPy, Pandas, etc.
- Nowadays, open source development is very common, and even commercial software development is involved with open source software.
  - e.g.) The dispute between Amazon and Elastic.
  - Undesirable trend: growing as an open source project → converting to limited commercial license.

License models

- ***The GNU General Public License (GPL)***.
  - This is a so-called ‘reciprocal’ license - if you use open source software licensed under GPL, then you must make that software open source.
- ***The GNU Lesser General Public License (LGPL)***
  - This is a variant of the GPL license where you can write components that link to open source code without having to publish the source of these components.
- ***MIT*** License.
  - This is a non-reciprocal license, allows you anything including commercial uses without much limitations, as long as you include the copy of the license.
  - You can include the code in proprietary systems that are sold.

License models

- Github License Doc
- https://choosealicense.com/
- You can find various license models.



License models

GNU LGPLv3

Permissions of this copyleft license are conditioned on making available complete source code of licensed works and modifications under the same license or the GNU GPLv3. Copyright and license notices must be preserved. Contributors provide an express grant of patent rights. However, a larger work using the licensed work through interfaces provided by the licensed work may be distributed under different terms and without source code for the larger work.

Permissions
- Commercial use
- Distribution
- Modification
- Patent use
- Private use

Conditions
- Disclose source
- License and copyright notice
- Same license (library)
- State changes

Limitations
- Liability
- Warranty

View full GNU Lesser General Public License v3.0 »

Mozilla Public License 2.0

Permissions of this weak copyleft license are conditioned on making available source code of licensed files and modifications of those files under the same license (or in certain cases, one of the GNU licenses). Copyright and license notices must be preserved. Contributors provide an express grant of patent rights. However, a larger work using the licensed work may be distributed under different terms and without source code for files added in the larger work.

Permissions
- Commercial use
- Distribution
- Modification
- Patent use
- Private use

Conditions
- Disclose source
- License and copyright notice
- Same license (file)

Limitations
- Liability
- Trademark use
- Warranty

View full Mozilla Public License 2.0 »

Summary

- We have discussed design and implementation issues.
- Architectural design is about the overall system structure, which requires high cost for refactoring.
- Software design and implementation activities are invariably inter-leaved.
- For architecture and software design, we may consider design patterns.
- Important implementation issues include reuse, configuration management, and host-target development.